Technical Requirement Document (TRD)

Personal Health Record Management and Sharing System (PHRMS)

Target Environment: Local Execution Prototype
Author: Abishekan Sri Baskaran (Student ID: 0001044)
Course: MIS 4500 – Technology Capstone Project (Fall 2026)

---

1. System Architecture Overview

The PHRMS prototype follows a modular **Model-View-Controller (MVC)** web architecture designed for reliable local execution and rapid demonstration during the capstone evaluation.

- **Presentation Layer (Frontend):** HTML5, CSS3, JavaScript (ES6+), and **Bootstrap 5** styled with custom CSS variables mapped to the approved Medical Color Palette.
- **Application Layer (Backend): Python 3.x** with the **Flask** micro-framework, handling session authentication, role-based access control (RBAC), file upload validation, state transitions, and audit logging.
- **Data Layer (Database & Storage): SQLAlchemy ORM** configured to run out-of-the-box with a local **SQLite** database (`phrms.db`) for zero-configuration demo portability, while remaining 100% compatible with **MySQL** (`mysql+pymysql://`) via a single configuration variable. Uploaded files are stored in a local `uploads/` directory with sanitized filenames.

2. Database Schema Design (Relational Model)

2.1 `users` Table

Stores all actors across the three stakeholder roles.

- `id` (INT, Primary Key, Auto-Increment)
- `full_name` (VARCHAR(120), Not Null)
- `email` (VARCHAR(120), Unique, Not Null)
- `password_hash` (VARCHAR(255), Not Null)
- `role` (ENUM: `'PATIENT'`, `'REVIEWER'`, `'DOCTOR'`, Not Null)
- `specialization` (VARCHAR(100), Nullable — used for `DOCTOR` role, e.g., “Cardiology”, “General Medicine”)
- `date_of_birth` (DATE, Nullable — used for `PATIENT` role)
- `blood_group` (VARCHAR(10), Nullable — used for `PATIENT` role)
- `known_allergies` (TEXT, Nullable — used for `PATIENT` role)
- `created_at` (DATETIME, Default `CURRENT_TIMESTAMP`)

2.2 `health_records` Table

Stores uploaded medical documents, manual health data entries, and administrative review states.

- `id` (INT, Primary Key, Auto-Increment)
- `patient_id` (INT, Foreign Key → `users.id`, Not Null)
- `title` (VARCHAR(150), Not Null)
- `category` (ENUM: `'Lab Report'`, `'Prescription'`, `'Consultation Note'`, `'Imaging / Scan'`, `'Vaccination / Vitals'`, Not Null)
- `provider_name` (VARCHAR(150), Not Null — Hospital, Clinic, or Lab name)
- `record_date` (DATE, Not Null — Date the medical event/report occurred, used for chronological sorting)
- `clinical_summary` (TEXT, Not Null — Manual health notes, key vitals, or findings entered by the patient)
- `file_path` (VARCHAR(255), Nullable — Path to uploaded PDF/Image in local storage)
- `file_name` (VARCHAR(255), Nullable — Original sanitized filename)
- `status` (ENUM: `'PENDING'`, `'APPROVED'`, `'CORRECTION_REQUIRED'`, `'REJECTED'`, Default `'PENDING'`)
- `reviewer_id` (INT, Foreign Key → `users.id`, Nullable)
- `review_notes` (TEXT, Nullable — Feedback from the Document Review Team)
- `reviewed_at` (DATETIME, Nullable)
- `submitted_at` (DATETIME, Default `CURRENT_TIMESTAMP`)

2.3 `doctor_access` Table

Manages patient-controlled authorization permissions (FR06, FR07).

- `id` (INT, Primary Key, Auto-Increment)
- `patient_id` (INT, Foreign Key → `users.id`, Not Null)
- `doctor_id` (INT, Foreign Key → `users.id`, Not Null)
- `is_active` (BOOLEAN, Default `True`)
- `access_notes` (VARCHAR(255), Nullable — e.g., “Authorized for Cardiology follow-up”)
- `granted_at` (DATETIME, Default `CURRENT_TIMESTAMP`)
- `revoked_at` (DATETIME, Nullable)

2.4 `consultation_summaries` Table

Stores consultation-ready summaries compiled by patients from selected approved records (FR08).

- `id` (INT, Primary Key, Auto-Increment)
- `patient_id` (INT, Foreign Key → `users.id`, Not Null)
- `title` (VARCHAR(150), Not Null — e.g., "Pre-Consultation Summary: Cardiology Follow-up")
- `chief_complaint` (TEXT, Not Null — Patient's reason for visit / current symptoms)
- `selected_record_ids` (JSON / TEXT, Not Null — List of `health_records.id` included in this summary)
- `created_at` (DATETIME, Default `CURRENT_TIMESTAMP`)

2.5 `audit_logs` Table

Tracks all security-sensitive and workflow events (FR09).

- `id` (INT, Primary Key, Auto-Increment)
- `actor_id` (INT, Foreign Key → `users.id`, Not Null)
- `patient_id` (INT, Foreign Key → `users.id`, Nullable — The patient whose data was affected/viewed)
- `action` (VARCHAR(100), Not Null — e.g., `'RECORD_SUBMITTED'`, `'RECORD_APPROVED'`, `'RECORD_CORRECTION_R`
  `'RECORD_REJECTED'`, `'ACCESS_GRANTED'`, `'ACCESS_REVOKED'`, `'DOCTOR_VIEWED_TIMELINE'`, `'SUMMARY_GENERA`
- `details` (TEXT, Not Null)
- `timestamp` (DATETIME, Default `CURRENT_TIMESTAMP`)

3. Application Routes & API Endpoints

Route	Method	Role Allowed	Description
/login	GET, POST	Public	Authenticates user credentials and initializes role session; includes quick-login demo buttons.
/logout	GET	Authenticated	Clears session and logs out user.
/patient/dashboard	GET	PATIENT	Displays summary metrics, submission status tracker, chronological timeline, and quick actions.
/patient/records/new	POST	PATIENT	Uploads a new medical document & metadata; sets status to PENDING and logs action.
/patient/records/<id>/resubmit	POST	PATIENT	Updates a record marked CORRECTION_REQUIRED and resets status to PENDING.
/patient/access	POST	PATIENT	Grants or revokes a specific doctor's access and writes an audit log entry.
/patient/summary/new	POST	PATIENT	Generates a new consultation-ready summary from selected APPROVED record IDs.
/reviewer/dashboard	GET	REVIEWER	Displays the administrative review queue (PENDING items) and recent review history.
/reviewer/records/<id>/decide	POST	REVIEWER	Submits decision (APPROVED, CORRECTION_REQUIRED, REJECTED) + reviewer notes.
/doctor/dashboard	GET	DOCTOR	Lists all patients who have actively authorized the logged-in doctor.
/doctor/patient/<patient_id>	GET	DOCTOR	Verifies active authorization, logs access, and renders the patient's timeline and summaries.
/summary/<summary_id>	GET	PATIENT, DOCTOR	Renders a clean, printable Consultation-Ready Summary sheet.

4. Security, Validation & Fictitious Data Seeding

1. **Role-Based Access Control (RBAC):** Every route is guarded by a `@role_required(...)` decorator that validates both login state and user role.
2. **Authorization Enforcement:** Before any `/doctor/patient/<patient_id>` request completes, the backend queries `doctor_access` for `(patient_id, doctor_id, is_active=True)`. Unauthorized requests are rejected with HTTP 403 and logged.
3. **File Handling:** Allowed file extensions are restricted to `.pdf`, `.png`, `.jpg`, and `.jpeg` using `werkzeug.utils.secure_filename`.
4. **Pre-Seeded Fictitious Dataset:** On first run, the database automatically seeds realistic fictitious personas (*Amara Perera* and *Rohan Silva* as Patients; *Dr. Nalin Bandara* and *Dr. Kavindi Jayasinghe* as Doctors; *Malini Fernando* as Document Review Team) along with sample lab reports, prescriptions, pending uploads, correction requests, and consultation summaries so all features can be demonstrated immediately.